VOCEL: Efficient Concept Learning with Learned Value and Ranking Guidance

Anonymous Author(s)

ACM Reference Format:
Anonymous Author(s). 2018. VOCEL: Efficient Concept Learning with Learned Value and Ranking Guidance. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Appendix

This appendix provides additional details that complement the main paper. Specifically, it includes implementation details, supplementary experimental results, additional ablation studies, and further analyses that were omitted from the main text due to space constraints. Unless otherwise stated, the notation and experimental setup follow those described in the main paper.

1.1 Problem Setting and Search Strategy

Given a knowledge base $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ and a target learning problem (E^+, E^-), VOCEL searches for an $\mathcal{ALC}$ class expression C that maximizes the F_1 score. Algorithm 2 structures this search by combining K-best priority search, structural and neural candidate pruning, and recursive covering for difficult LPs.

1.1.1 K-Best Batch Search. The algorithm initializes the search at the top concept $\top$ (lines 1–4). Candidates are maintained in a priority queue $\mathcal{H}$ organized as a min-heap indexed by $-s(C)$, where the score function balances accuracy and complexity:

$$s(C) = F_1(C) - \lambda \cdot |C| \tag{1}$$

Here, λ acts as a length penalty parameter. A visited set prevents redundant evaluations, while a subproblem set $\mathcal{K}$ tracks previously encountered recursive subproblems.

Rather than expanding a single concept per iteration, VOCEL pops up to b best-performing concepts from $\mathcal{H}$ into a batch $\mathcal{B}$ during each round (lines 5–9), bounded by a maximum pop budget N. This K-best beam mechanism allows the search to explore multiple promising search trajectories in parallel.

1.1.2 Concept Expansion & Filtering. For each concept $C \in \mathcal{B}$, candidate refinements $D \in \rho(C)$ are generated (lines 10–17). For every unvisited candidate:

- Exact coverage metrics—$F_1(D)$, Precision $\mathrm{Prec}(D)$, and true positive count $\mathrm{tp}(D)$—are computed.

Algorithm 1: Offline Training of Concept-Level Guidance

Input: Positive examples E^+, negative examples E^-, refinement operator ρ, beam width b, top-k concepts per expansion k, maximum depth $depth_{\max}$, time budget T, aggregation function μ_{agg}, learning rate η

Output: Trained value model $\hat{V}_\theta$

1 **Phase 1: Data Generation (Beam Search)**
2 $\mathcal{D} \leftarrow \{(\top, I(\top), F_1(\top), \text{null}, 0)\}$;
3 $B_0 \leftarrow \{\top\}$, Visited $\leftarrow \{\top\}$;
4 **for** $depth \leftarrow 0$ **to** $depth_{\max} - 1$ ***and*** $time < T$ **do**
5 $G \leftarrow \emptyset$;
6 **foreach** $C \in B_{depth}$ **do**
7 **foreach** $D \in \rho(C)$ **do**
8 **if** $D \notin$ Visited **then**
9 Visited$\cup = \{D\}$;
10 $G \cup = \{(D, F_1(D), I(D), C)\}$;
11 sort G by decreasing F_1;
12 **for** $i \leftarrow 1$ **to** k **do**
13 add $(D, I(D), F_1(D), C, depth + 1)$ from G_i to $\mathcal{D}$;
14 $B_{d+1} \leftarrow$ top-b concepts from G;
15 **Phase 2: Best-Reachable Label Propagation**
16 **foreach** $C \in \mathcal{D}$ **do**
17 $y_C \leftarrow F_1(C)$;
18 **for** $C \in \mathcal{D}$ *sorted by decreasing depth* **do**
19 **foreach** $D \in \text{children}(C)$ **do**
20 $y_C \leftarrow \max(y_C, y_D)$;
21 **Phase 3: Feature Construction and Training**
22 **foreach** $C \in \mathcal{D}$ **do**
23 $\mu_C \leftarrow \mu(I(C))$;
24 $\mu^+ \leftarrow \mu_{agg}(E^+)$;
25 $\mu^- \leftarrow \mu_{agg}(E^-)$;
26 **foreach** $C \in \mathcal{D}$ **do**
27 $x_C \leftarrow [\mu_{agg}(C); \mu^+; \mu^-]$;
28 add (x_C, y_C) to $\mathcal{T}$;
29 Train $\hat{V}_\theta$ on $\mathcal{T}$;
30 using MSE or rank + Adam(η);
31 **return** $\hat{V}_\theta$;

- If a perfect concept ($F_1(D) = 1$) is found, the search returns D immediately.
- A local skip rule acts as a lightweight structural filter: candidate D is retained in the round's survivor set $\mathcal{S}$ only if its F_1 score matches or improves upon the local best score

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Algorithm 2: VOCEL: K-Best Priority Search

Input: Positive examples E^+, negative examples E^-, refinement operator ρ, beam width b, pop budget N, length penalty λ, V-Net $\hat{V}_\theta$, precision threshold θ_p
Output: Best concept hypothesis C^*

1 Initialize priority queue $\mathcal{H} \leftarrow \{\top\}$;
2 Initialize Visited $\leftarrow \{\top\}$,;
3 Initialize best solution $C^* \leftarrow \top$;
4 Initialize covered subproblems $\mathcal{K} \leftarrow \emptyset$;
5 **while** $\mathcal{H} \neq \emptyset$ ***and*** *pop budget not exhausted* **do**
6 $\quad \mathcal{B} \leftarrow \text{ExtractTop}(\mathcal{H}, b)$;
7 $\quad \mathcal{S} \leftarrow \emptyset$;
8 $\quad$ **foreach** $C \in \mathcal{B}$ **do**
9 $\quad\quad$ Initialize local best score ℓ_{best};
10 $\quad\quad$ **foreach** $D \in \rho(C) \setminus$ Visited **do**
11 $\quad\quad\quad$ Visited $\leftarrow$ Visited $\cup \{D\}$;
12 $\quad\quad\quad$ Evaluate D (compute F_1, precision and covered positives);
13 $\quad\quad\quad$ **if** $F_1(D) = 1$ **then**
14 $\quad\quad\quad\quad$ **return** D
15 $\quad\quad\quad$ UpdateBest(D);
16 $\quad\quad\quad$ **if** *Keep*(D, ℓ_{best}) **then**
17 $\quad\quad\quad\quad \mathcal{S} \leftarrow \mathcal{S} \cup \{D\}$;
18 $\quad$ **if** $\hat{V}_\theta$ *is available* **then**
19 $\quad\quad \mathcal{S} \leftarrow \text{VNetPrune}(\mathcal{S}, \hat{V}_\theta)$;
20 $\quad$ **foreach** $D \in \mathcal{S}$ **do**
21 $\quad\quad$ Push D into $\mathcal{H}$ with priority $F_1(D) - \lambda|D|$;
22 $\quad$ **foreach** $D \in \mathcal{S}$ *satisfying the covering criterion* **do**
23 $\quad\quad$ **if** *Residual subproblem not solved before* **then**
24 $\quad\quad\quad D_{\text{sub}} \leftarrow \text{VOCEL}(E^+ \setminus I(D), E^-)$;
25 $\quad\quad\quad D_{\text{comb}} \leftarrow D \sqcup D_{\text{sub}}$;
26 $\quad\quad\quad$ Evaluate D_{comb};
27 $\quad\quad\quad$ UpdateBest(D_{comb});
28 $\quad\quad\quad$ Push D_{comb} into $\mathcal{H}$;
29 **return** C^*

observed in that expansion branch (ℓ_{best}), or if D introduces a new existential quantifier ($\exists r$).

1.1.3 V-Net Neural Batch Pruning. Once the initial survivors are collected into $\mathcal{S}$, VOCEL applies neural pruning (lines 18–19). If a trained value network $\hat{V}_\theta$ is available and multiple survivors exist:

- The model computes predicted potential values $\hat{\mathbf{v}}$ based on embedding representations of D, E^+, and E^-.
- The algorithm calculates the batch mean predicted value $\bar{v} = \text{mean}(\hat{\mathbf{v}})$.
- Only candidates whose predicted potential meets or exceeds the batch mean ($\hat{v}_D \geq \bar{v}$)—or candidates that already exceed the global best score ($F_1(D) \geq F_1^*$)—are retained in $\mathcal{S}$.

1.1.4 Inline Recursive Covering. Surviving concepts $(D, f, p, t) \in \mathcal{S}$ are evaluated using the penalized score function $s(D) = f - \lambda \cdot |D|$ and pushed back into the priority queue $\mathcal{H}$ for future expansion (lines 20–21).

To naturally construct disjunctive concepts (e.g., $D \sqcup D_{\text{sub}}$), VOCEL evaluates highly precise survivors (lines 22–28):

- If a survivor candidate D satisfies precision $\text{Prec}(D) \geq \theta_p$ and covers at least $n_{\min}$ positive examples, VOCEL isolates the remaining uncovered positives $E^+ \setminus E^{+\prime}$, where $E^{+\prime} = I(D) \cap E^+$.
- If this residual set forms a new subproblem key $\mathbf{k} \notin \mathcal{K}$, VOCEL recursively calls itself to learn a hypothesis for the uncovered positives: $D_{\text{sub}} \leftarrow \text{VOCEL}(E^+ \setminus E^{+\prime}, E^-)$.
- It constructs the combined disjunctive concept $D_{\text{comb}} = D \sqcup D_{\text{sub}}$, checks for early termination ($F_1 = 1$), updates C^*, and pushes D_{comb} into $\mathcal{H}$.

1.1.5 Search Termination. The algorithm terminates when either a concept with $F_1 = 1$ is discovered, the priority queue $\mathcal{H}$ becomes empty, or the maximum pop budget N is reached, returning the overall best concept hypothesis C^*.

1.2 Value Network

The value function $\hat{V}_\theta$ is implemented as a neural network operating on set-based representations of concepts. Each entity e (i.e., an individual in the knowledge base) is associated with a pre-trained embedding $\phi(e) \in \mathbb{R}^d$. The value network predicts $\hat{V}_\theta(C) = f_\theta(\mathbf{x}_C)$, which approximates the best reachable future F1 score from C. Intuitively, rather than evaluating only the current quality of C, the model estimates how good the best refinement of C could eventually become. For instance, consider the *Aunt* learning problem from the Family dataset. Starting from a general concept, the learner incrementally refines candidate expressions. At each step, the value network assigns a score to guide the search toward concepts with high potential. Even if an intermediate concept has a moderate current F1 score, it may still receive a high predicted value if it can be refined into a high-quality concept. At every refinement step, top-2 high predicted reachable F1 score are further refined while less promising nodes are discarded (See Figure 1).

In this example, the search is guided toward the concept $\text{Female} \sqcap \Big((\exists \text{married}.(\exists \text{hasSibling}.(\exists \text{hasChild}.\top))) \sqcup (\exists \text{hasSibling}.(\exists \text{hasChild}.\top))\Big)$ which indeed describes an *Aunt*.

Data Generation. To obtain supervised training data for the offline variants, we proceed as in the current literature [**?**] by constructing LPs directly from named classes in each knowledge base. For each named class, its asserted individuals are used as positive examples, while the remaining individuals are used as negative examples.

For each LP, we perform a bounded best-first search starting from $\top$ using the same refinement operator as the inference procedure. At each iteration, the highest-scoring b candidates are expanded, where $b \in \{5, 10\}$ depending on the dataset. Search is bounded by a maximum refinement depth of 15 and a per-LP time budget ranging from 60 to 180 seconds depending on the size of the dataset. Each expanded node records the candidate class expression, its F1 score on the LP, its quantifier depth, a pointer to its parent, and its covered instances. Recursive union steps are retained in the trace

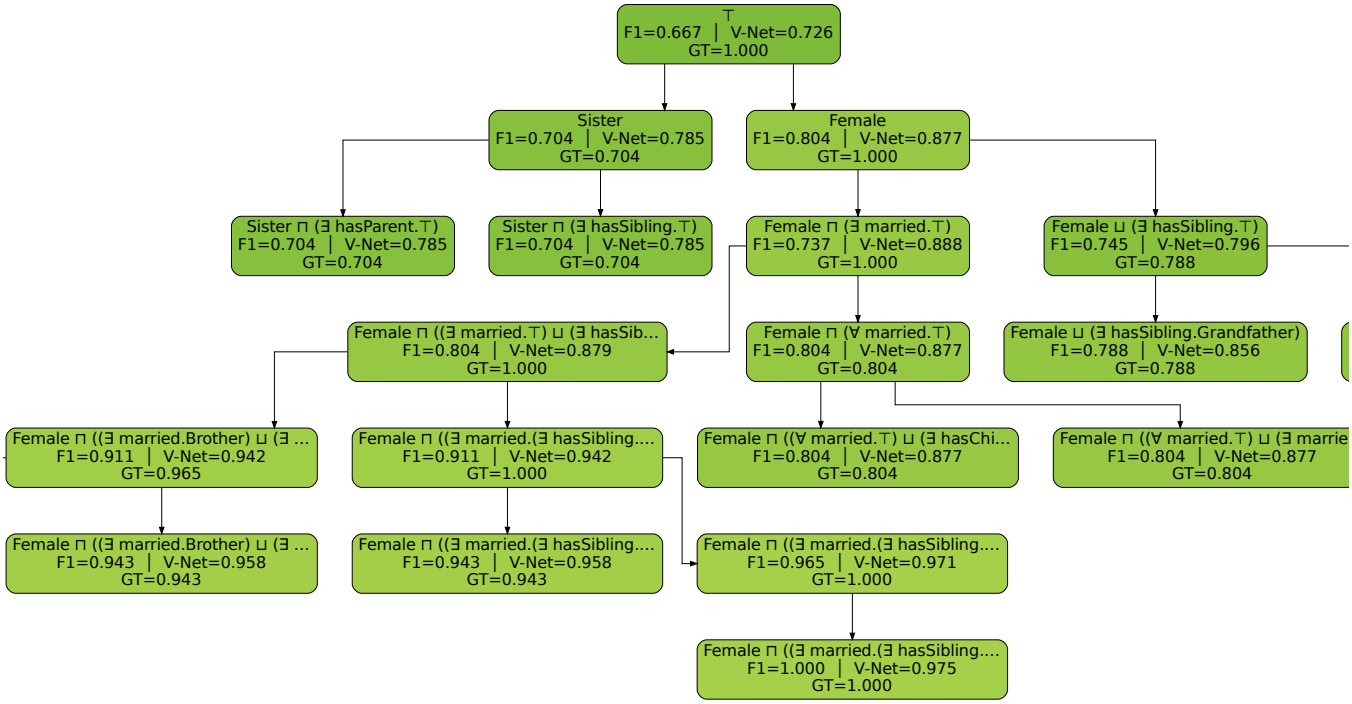


Figure 1: Illustration of value-based search in concept refinement on the *Aunt* learning problem of the Family dataset. Each node represents a concept annotated with its F1 score. GT represents the true best reachable F1 score of the concept. The value network estimates the best achievable F1 (V-Net) from intermediate concepts, guiding the search toward promising concepts. A large gap between V-Net and GT means the model is uncertain about that branch. A node where V-Net ≈ GT is one that the model has learned to value correctly.

so that concepts reachable through disjunctive compositions are represented.

The overall procedure is described in Algorithm 1 (lines 1–14). Starting from the top concept, the search iteratively expands candidates using the refinement operator ρ, while maintaining a beam of size b. At each depth, newly generated concepts are evaluated using F_1 and inserted into a candidate pool (lines 6–11). The top-k concepts are then recorded into the dataset $\mathcal{D}$ (lines 12–14), resulting in

$$\mathcal{D} = \{(C, I(C), F_1(C), \text{parent}(C), \text{depth}(C))\}. \tag{2}$$

In total, this procedure yields 2,331 learning problems and approximately 23,000 recorded search nodes across the six benchmark datasets.

Target Construction. To obtain supervision signals, we compute for each concept C its best reachable F1 score using a bottom-up propagation procedure (Algorithm 1, lines 17–20). Initially, each concept is assigned its immediate score $F_1(C)$ (line 17). Then, processing concepts in decreasing depth order, values from children are propagated upward (lines 18–20), yielding

$$y_C = \max_{D \text{ reachable from } C} F_1(D). \tag{3}$$

subsection Further Results

Figure 2 shows the evolution of the training loss across datasets and Figure 3 shows the training time on all datasets. Overall, the value network shows stable convergence, with a rapid decrease in loss during the early epochs followed by a gradual plateau. This indicates that the network quickly captures distinctions between promising and non-promising concepts, and then refines its predictions over time. We observe some variability across datasets: datasets such as Lymphography and Mutagenesis converge faster and achieve lower loss values, whereas others, such as Animals or Carcinogenesis, converge more slowly and exhibit higher residual error. Importantly, no signs of divergence or instability are observed, suggesting that the combination of embedding representations and optimization is sufficient to learn a consistent value approximation across all datasets. In Figure 3, training times are in general low for most VOCEL variants. VOCEL-BS trains in under 2.5 minutes across all datasets, and VOCEL-RK remains competitive at under 4 minutes, making both variants considerably faster than Drill, which requires up to 14 minutes on Carcinogenesis. The heavier aggregation modules of VOCEL^{DS} and VOCEL-BSST increase training time by a factor of 3-6× relative to VOCEL-BS, yet they still match or undercut Drill on most datasets, demonstrating that the additional architectural complexity comes at a modest and acceptable cost.

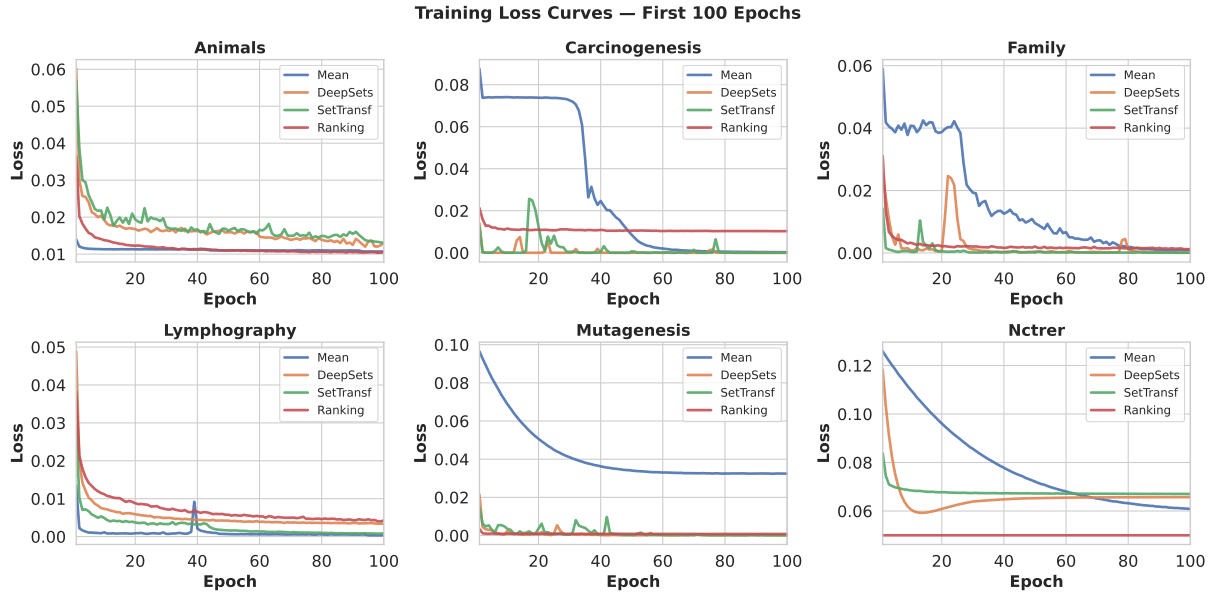


Figure 2: V-Net training loss across all datasets.

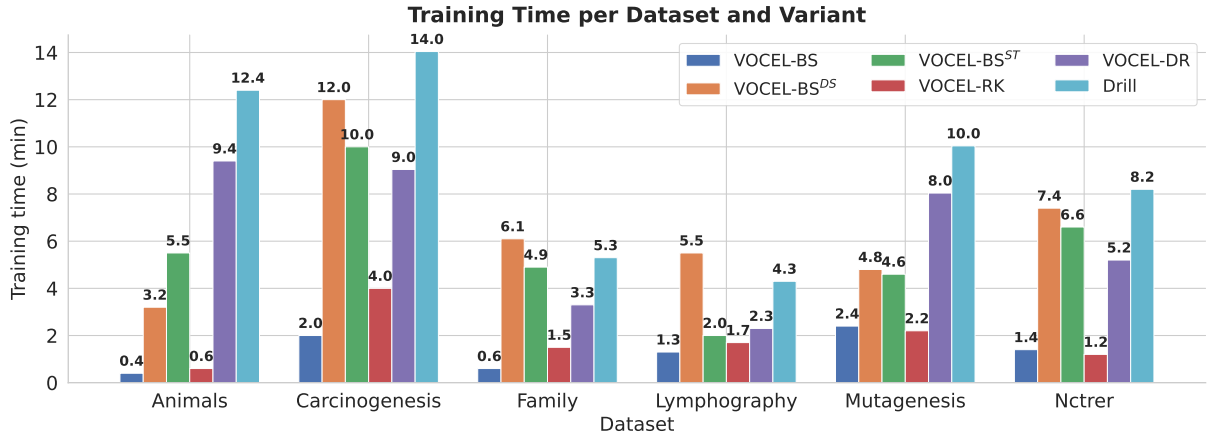


Figure 3: V-Net training loss across all datasets.

1.3 Statistical Analysis

To statistically validate the reduction in search effort, we performed a Wilcoxon signed-rank test on the pooled results across all datasets and variants for the number of tested concepts. As shown in Table 2, VOCEL-BS consistently explores significantly fewer concepts than all compared baselines, with p-values below 10^{-6} in all cases and consistently large effect sizes. The median reduction ranges from 19 concepts against PruneCEL to 9,973 against DRILL, demonstrating substantial gains in search efficiency. Furthermore, the results also show that VOCEL-DR explores significantly fewer concepts than the original DRILL.The Table also provides the Wilcoxon signed-rank test results for the F1 score and runtime. We also investigate this among VOCEL variants as shown in Table 1. From that Table, we can observe that in general, VOCEL-RK significantly explores fewer concepts than other variants, which means that ranking is more efficient in selecting concepts to guide further refinements. The Table also shows that VOCEL-BS is, in general, significantly faster at inference time compared to other variants.

V1	V2	# Concepts		F1-score		Runtime	
		p_{bonf}	Winner	p_{bonf}	Winner	p_{bonf}	Winner
VOCEL-BS	VOCEL-BS ^{DS}	1.000	ns	1.000	ns	$< 10^{-6}$	VOCEL-BS
VOCEL-BS	VOCEL-BS ST	1.000	ns	1.000	ns	1.000	ns
VOCEL-BS	VOCEL-RK	$< 10^{-6}$	VOCEL-RK	1.000	ns	$< 10^{-6}$	VOCEL-BS
VOCEL-BS	VOCEL-DR	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	VOCEL-BS	0.353	ns
VOCEL-BS ^{DS}	VOCEL-BS ST	0.394	ns	1.000	ns	1.000	ns
VOCEL-BS ^{DS}	VOCEL-RK	0.135	ns	1.000	ns	$< 10^{-6}$	VOCEL-BS ^{DS}
VOCEL-BS ^{DS}	VOCEL-DR	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	VOCEL-BS ^{DS}	0.464	ns
VOCEL-BS ST	VOCEL-RK	0.002	VOCEL-RK	1.000	ns	0.117	ns
VOCEL-BS ST	VOCEL-DR	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST	0.014	VOCEL-DR
VOCEL-RK	VOCEL-DR	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	VOCEL-RK	0.688	ns

Table 1: Pairwise Wilcoxon signed-rank tests between VOCEL variants with Bonferroni correction. The winner column indicates the better-performing variant (higher F1-score; fewer generated concepts and lower runtime). "ns" denotes a non-significant difference.

VOCEL	Baseline	# Concepts		F1-score		Runtime	
		p_{bonf}	Winner	p_{bonf}	Winner	p_{bonf}	Winner
VOCEL-BS	CELOE	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	VOCEL-BS	0.053	ns
	OCEL	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	VOCEL-BS	1.000	ns
	EvoLearner	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	EvoLearner
	ALCSAT	ns	ns	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	ALCSAT
	SPELL	ns	ns	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	SPELL
	NERO	ns	ns	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	NERO
	Drill	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	VOCEL-BS	1.000	ns
	PruneCEL	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	VOCEL-BS	$< 10^{-6}$	VOCEL-BS
VOCEL-BS ^{DS}	CELOE	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	VOCEL-BS ^{DS}	0.168	ns
	OCEL	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	VOCEL-BS ^{DS}	1.000	ns
	EvoLearner	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	EvoLearner
	ALCSAT	ns	ns	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	ALCSAT
	SPELL	ns	ns	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	SPELL
	NERO	ns	ns	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	NERO
	Drill	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	VOCEL-BS ^{DS}	1.000	ns
	PruneCEL	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	VOCEL-BS ^{DS}	$< 10^{-6}$	VOCEL-BS ^{DS}
VOCEL-BS ST	CELOE	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST	0.526	ns
	OCEL	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST
	EvoLearner	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST	1.000	ns
	ALCSAT	ns	ns	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	ALCSAT
	SPELL	ns	ns	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	SPELL
	NERO	ns	ns	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	NERO
	Drill	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST
	PruneCEL	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST	$< 10^{-6}$	VOCEL-BS ST
VOCEL-RK	CELOE	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	VOCEL-RK	0.395	ns
	OCEL	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	VOCEL-RK	1.000	ns
	EvoLearner	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	EvoLearner
	ALCSAT	ns	ns	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	ALCSAT
	SPELL	ns	ns	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	SPELL
	NERO	ns	ns	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	NERO
	Drill	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	VOCEL-RK	1.000	ns
	PruneCEL	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	VOCEL-RK	$< 10^{-6}$	VOCEL-RK
VOCEL-DR	CELOE	$< 10^{-6}$	VOCEL-DR	1.000	ns	$< 10^{-6}$	CELOE
	OCEL	$< 10^{-6}$	VOCEL-DR	0.028	OCEL	1.000	ns
	EvoLearner	$< 10^{-6}$	VOCEL-DR	0.187	ns	1.000	ns
	ALCSAT	ns	ns	$< 10^{-6}$	VOCEL-DR	$< 10^{-6}$	ALCSAT
	SPELL	ns	ns	$< 10^{-6}$	VOCEL-DR	$< 10^{-6}$	SPELL
	NERO	ns	ns	$< 10^{-6}$	VOCEL-DR	$< 10^{-6}$	NERO
	Drill	$< 10^{-6}$	VOCEL-DR	$< 10^{-6}$	VOCEL-DR	$< 10^{-6}$	VOCEL-DR
	PruneCEL	$< 10^{-6}$	VOCEL-DR	$< 10^{-6}$	VOCEL-DR	$< 10^{-6}$	PruneCEL

Table 2: Wilcoxon signed-rank tests with Bonferroni correction over all learning problems and datasets. The winner column indicates the better-performing method (higher F1-score; fewer generated concepts and lower runtime). "ns" denotes a non-significant difference.